Reliable Data Transfer over UDP

CS-30003 · Coding Assignment 1 · Catalogue Project P3 (Path A) · Team Plan

Duration 8 weeks · Sep 1 – Oct 26, 2026 **Marks** 10 (Proposal 2 · Implementation 3 · Viva 5) **Team** 4 members

Stack Java 21 + Python (analysis) on WSL2

1. What we are building, in one paragraph

A file-transfer system over UDP with three interchangeable ARQ transports — Stop-and-Wait, Go-Back-N, and Selective Repeat — behind one application interface. Alongside it we write our own channel emulator: a separate process that sits on the path and injects loss, duplication, reordering, corruption, and delay with jitter, all deterministic from a seed. The protocols are the system; the emulator is how we measure it. We then run four experiments and defend the numbers.

2. The three rules that decide our marks

RULE FROM THE BRIEF	WHAT IT MEANS FOR US DAY TO DAY
Implement the protocol, not call it	Sockets only. `DatagramSocket` / `DatagramChannel` and nothing above them. No Netty, Aeron, KryoNet, JGroups, or any QUIC library. No TCP anywhere on the data path.
Produce evidence	Every claim needs a plot with error bars behind it. Ten seeds per data point, mean with 95% confidence intervals. A working demo alone scores badly.
Explain every line	The viva is **individual** and worth 5 of the 10 marks. Sir will ask you to modify your code live. Code that runs but cannot be explained scores **zero**.

> **The viva is worth more than the code.** Half the total marks are awarded to you individually, not to the group. This is why the split below gives everyone their own protocol to own, why we cross-teach in Week 6, and why nobody is allowed to be "the person who only makes the graphs." You will each be asked about parts of the system you did not write.

3. How the work divides — four equal roles

Each role owns two core components, one experiment, one section of the final report, and the tests for their own code. Estimated effort is deliberately balanced within a few hours.

ROLE	CORE COMPONENTS OWNED	EXPERIMENT OWNED	REPORT SECTION	EST.
M1 **Framing & Session**	`Packet`, metadata handshake, `StopAndWait`, sender/receiver CLIs, FIN teardown	Exp 1 — goodput vs loss rate	Wire format & session	46 h
M2 **Timers & Go-Back-N**	`TimerWheel`, `RttEstimator` (Jacobson/Karels + Karn), `GoBackN`	Exp 4 — RTO sensitivity	Timers, RTO, GBN	48 h
M3 **Selective Repeat**	`SelectiveRepeat`, receiver buffer, SACK encode/decode, wraparound tests	Exp 2 — goodput vs window	Selective Repeat	50 h
M4 **Channel & Evidence**	`NetEm` emulator, JSONL trace, `RunStats`, experiment harness, plots	Exp 3 — reordering	Methodology & emulator	48 h

Shared by everyone, non-negotiable

- Write the tests for your own module. Nobody tests your code for you.

- Review one partner's pull request each week. Pairing rotates: odd weeks M1↔M2 and M3↔M4, even weeks M1↔M3 and M2↔M4.
- Take your turn as weekly integration owner (rotation in §5).
- Commit under your own name and email, at least three commits a week. Sir treats commit history as evidence and has said a repo with three commits in the final week signals a problem.
- Attend the Week 6 cross-teaching session and the Week 8 viva drills.

4. Role detail

M1 Framing & Session

Member: **Shaili Seth** · Roll: **2405901** · Section CSE-14

- **Rewrite `Packet.java` yourself** — 20-byte header, RFC 1071 checksum. A draft plus 13 passing tests already exist; the tests are your specification, the draft is reference only. See the AI-use note in §9.
- Unsigned handling: `seq` / `ack` are unsigned 32-bit on the wire and must be widened with `Integer.toUnsignedLong`. This is the single most common Java bug in this project.
- Metadata handshake: `seq=0` carries filename, size, SHA-256; receiver ACKs; data starts at `seq=1`.
- `StopAndWait.java` — the simplest protocol, and the one that proves the framing works.
- `Sender.java` / `Receiver.java` command-line front ends.
- FIN / FINACK teardown; SHA-256 integrity comparison at the end of every transfer.
- **Experiment 1**: goodput versus loss rate, all three protocols at W=32.

M2 Timers & Go-Back-N

Member: **Sinjan Mishra** · Roll: **2405910** · Section CSE-14

- `TimerWheel.java` — a `PriorityQueue` of deadlines from `System.nanoTime()`, driven by one selector loop. **Single-threaded on purpose.** Do not use `ScheduledExecutorService` or `Timer`: they reintroduce exactly the concurrency the brief warns about, and they are harder to defend at viva.
- `RttEstimator.java` — Jacobson/Karels SRTT and RTTVAR, with Karn's algorithm (never sample RTT from a retransmitted packet) and exponential backoff.
- `GoBackN.java` — cumulative ACKs, one timer on the base of the window, retransmit the whole window on timeout.
- **Experiment 4**: RTO sensitivity. Fixed RTO at 1.2/1.5/2/3× RTT versus adaptive. Use the emulator trace as ground truth to label each retransmission *spurious* or *necessary* — this is the experiment no other group will have.

M3 Selective Repeat

Member: **Sohini Pandit** · Roll: **2405913** · Section CSE-14

- `SelectiveRepeat.java` — per-packet timers, receiver buffer holding out-of-order packets, in-order delivery to the application. **This is the hardest single module**, which is why it is the only one you own; start reading in Week 2, not Week 4.
- SACK block encoding and decoding. The `sackCount` header field is already reserved, so no wire-format change is needed.
- Window-bound correctness: prove Go-Back-N needs $W \le 2^k - 1$ and Selective Repeat needs $W \le 2^{k-1}$. Shrink the sequence space in a test, force wraparound, and demonstrate that an *illegal* window actually corrupts the transfer. Most groups never test this.
- **Experiment 2**: goodput versus window size, GBN against SR, at 1% and 5% loss.
- Week 7 stretch goal: SACK blocks in the live protocol.

M4 Channel & Evidence

Member: **Sohan Mandal** · Roll: **2405912** · Section CSE-14

- `NetEm.java` — a UDP middlebox process: sender → emulator → receiver and back, each direction impaired independently. Configurable loss, duplication, corruption, reordering, and delay with jitter.
- Seed with `java.util.Random(seed)`. Its algorithm is fixed by the Java specification, so the same seed reproduces byte-identical runs on any machine. **Never** `ThreadLocalRandom` or `SecureRandom`.
- JSONL impairment trace — one line per packet decision. This is what makes M2's Experiment 4 possible, and it lets us replay any strange run packet by packet at the viva.
- **Week 1 capacity calibration**: measure the emulator's maximum sustained packets/second. Every experiment must then run at under half of it, and the number goes in the report.
- `RunStats` → CSV, `run_matrix.py` sweep runner, `plots.py`.
- `tc netem` cross-validation in Week 6 — independent proof our emulator is honest.
- **Experiment 3**: retransmission count versus reordering probability.

M4 is not the graphs person. The emulator is a core component every other module depends on, and the analysis pipeline is where the implementation marks for "experiments, plots, and analysis" are actually earned. M4 will still be asked about Selective Repeat at the viva — which is what Week 6 cross-teaching is for.

5. Week by week

Every week ends at a **gate**: a binary, testable condition. If a gate is missed, the fallback in §7 fires that week, not in October.

WK	DATES	TARGETS	GATE	INTEGRATION OWNER
0	Aug 25–31	Names onto roles. Proposal finished and submitted. Repo shared with sir. JDK 21 installed inside WSL2 Ubuntu. Everyone runs `./build.sh` successfully.	Proposal submitted; build green on all four machines	M4
1	Sep 1–7	M1 rewrite `Packet`, handshake · M4 emulator v0 (loss only) + calibration · M2 read RFC 6298 · M3 read SR chapter	8 MiB transfer, SHA-256 match at 0% loss; calibration number committed	M1
2	Sep 8–14	M4 emulator v1: dup, corrupt, delay+jitter, reorder, JSONL trace · M2 `TimerWheel` · M1 Stop-and-Wait hardened + CLIs	SHA-256 match at 10% loss across 10 seeds	M2
3	Sep 15–21	M2 `GoBackN` + `RttEstimator` · M4 `RunStats` CSV + `run_matrix.py` · M3 SR design + receiver buffer started	GBN integrity across 5 channel profiles × 10 seeds; first real plot exists	M3
4	Sep 22–28	M3 `SelectiveRepeat` complete, wraparound tests · M2 adaptive RTO wired into all three protocols	**Core functionally done.** All three protocols pass one shared integrity suite	M4
5	Sep 29–Oct 5	Experiments 1, 2, 3 at full seed count. Report methodology section drafted.	Figures 1–3 regenerated by script from committed CSVs	M1
6	Oct 6–12	Experiment 4 with spurious-retransmit labelling · `tc netem` cross-validation · **cross-teaching session: each member teaches their module to the other three**	Figure 4 done; every member can trace a packet end to end through code they did not write	M2
7	Oct 13–19	One stretch goal only — SACK blocks (M3). Full report draft.	Report draft read and annotated by all four	M3
8	Oct 20–26	Code freeze Oct 22. Final report. Viva drills against each other.	`make all` reproduces every figure from a clean clone	M4

> **Critical path.** Everything downstream waits on two things: M1's `Packet` in Week 1 and M4's emulator in Week 2. If either slips, the whole schedule slips. These two people should pair in Week 1 if needed — the other two have reading to do and can absorb the delay.

6. The claim and the four experiments

Our claim: Selective Repeat sustains higher goodput than Go-Back-N under loss, and the gap widens with both loss rate and window size — Go-Back-N degrades roughly as $(1-p)^W$ where Selective Repeat degrades as $(1-p)$, while Stop-and-Wait is capped at MSS/RTT regardless of loss.

This is falsifiable because it predicts a *shape*, not just an ordering. If the measured curve does not match, we say so and explain why — that analysis earns more than a curve that happens to fit.

EXP	OWNER	VARIABLE	LEVELS	MEASURED
1	M1	Loss rate p	0, 0.1, 0.5, 1, 2, 5, 10, 20 %	Goodput, all three protocols, W=32
2	M3	Window size W	1, 2, 4, 8, 16, 32, 64, 128	Goodput, GBN vs SR, at p=1% and 5%
3	M4	Reorder probability	0, 1, 2, 5, 10, 20 %	Retransmission count, all three
4	M2	RTO policy	fixed 1.2/1.5/2/3×RTT vs adaptive	Spurious-retransmit fraction, total time

Constants across every run: 8 MiB file, 1400-byte payload, 10 seeds per cell, mean with 95% confidence intervals, all runs on one designated machine whose spec goes in the report, JVM warm-up transfer discarded before timing. Report goodput and wire throughput *separately* so retransmission overhead is visible rather than hidden.

7. Risks and what we do about them

RISK	WHY IT HAPPENS	FALLBACK, AND WHEN WE TAKE IT
Selective Repeat overruns Week 4	Receiver buffer plus per-packet timers is the hardest piece in the project.	Ship SR with a single timer on the oldest unacked packet. Still correct, still beats GBN. Document the simplification. **Decide on Sep 25**, not in October.
JVM timing jitter pollutes results	JIT compilation makes the first few thousand packets slow; GC pauses look exactly like network delay in a latency measurement.	Discard a warm-up transfer per JVM · pin the heap with `-Xms512m -Xmx512m` · keep the hot path allocation-free · log GC with `-Xlog:gc` and state in the report that no collection occurred during timed runs.
Our submission looks like another group's	P3 is a popular choice, and sir has said identical-looking submissions will be treated as collusion.	Three things make ours structurally different: the emulator runs as a *separate process* rather than an in-process hook, it writes a *replayable trace*, and Experiment 4 labels retransmissions spurious using that trace as ground truth. Write our own report prose. Share no code or figures with other groups, ever.
Someone falls behind quietly	Eight weeks is long enough to hide a stall for a month.	Weekly gates are binary and public. The integration owner reports status every week. Raise a problem in week 3, not week 7 — it is fixable in week 3.

8. Ground rules

Allowed

- `java.net`, `java.nio`, `java.util.concurrent`
- `MessageDigest` for SHA-256
- Python, pandas, matplotlib, numpy — analysis only
- Testing frameworks and build tooling

Banned

- Netty, Aeron, KryoNet, JGroups, any QUIC library
- Anything implementing reliable transport over UDP
- TCP anywhere on the data path

The rule in one line, from the brief: libraries may support your work; they may not *be* your work. If you are unsure whether something crosses the line, ask sir before you build on it. Asking is free; being found out at the demo is not.

Git

- One repository, shared with sir at proposal time.
- Everyone commits under their own name and email. Committing on someone else's behalf counts as **non-contribution by the absent member**.
- Small, frequent commits. At least three a week each.
- Branch per feature, pull request reviewed by your rotation partner before merge.

> **The contributor list must show exactly the four of us — nobody else.** Sir monitors the repository as we go, and a bot in the contributor graph invites exactly the questions we do not want. So: **do not enable Dependabot** (check Settings → Code security, it can be on by default), do not add a `.github/dependabot.yml`, do not merge bot pull requests, and never put an AI co-author trailer in a commit message. If you use an AI assistant, it is disclosed in `AI-USE.md` — that is the sanctioned channel, and git metadata is not.

9. AI tools — read this carefully

AI assistants are permitted with conditions, and we maintain `AI-USE.md` in the repository recording what we used them for. Honest disclosure carries no penalty; undisclosed use found at the viva does.

Permitted

- Explaining concepts
- Debugging
- Reviewing our code
- Generating tests
- Boilerplate and plotting scripts
- Improving our writing

Not permitted

- Generating our core protocol implementation and submitting it as our work

Our team rule

All four of us have an AI assistant of some kind, so this is worth saying out loud rather than assuming: **you may use AI to understand your module, but you write the code.**

Two reasons, both about marks rather than principle. The viva is individual and worth 5 of the 10 marks, and code you did not write is code you cannot modify under pressure when sir asks you to change it live — that scores zero however well it runs. And a chatbot asked to write Go-Back-N in Java gives roughly the same answer to everybody, which walks straight into the collusion rule.

> **Enforcement, at pull request time.** Before approving, the reviewer asks two or three "why" questions about the code. *Why this data structure? What happens if this ACK is lost? Why this bound and not that one?* If the author cannot answer, it does not merge. Five minutes per PR, it rehearses the viva every single week, and it catches copy-pasted code in September while there is still time to fix it.

> **Two open items before submission.** Draft versions of `Packet.java` (M1) and the channel emulator (M4) in the repository were AI-generated, and both are core deliverables that must be rewritten by hand. The emulator is the stricter of the two: the catalogue entry does not merely permit us to write it, it requires "*a channel emulator **you write yourself***" in those words. The tests stay in both cases — generating tests is explicitly permitted, and they are what tell you the rewrite is correct. Both are tracked as unticked boxes in `AI-USE.md`.

10. Viva preparation

Week 8, run these at each other until nobody hesitates. Sir has said he will ask you to modify your code live during the demo.

- Explain the Internet checksum byte by byte. Why one's complement? Why end-around carry?
- Why does Selective Repeat need $W \le 2^{k-1}$ when Go-Back-N only needs $W \le 2^k - 1$?
- Why `Integer.toUnsignedLong` on the sequence field? What breaks without it?
- How do you avoid a race between a timeout firing and an ACK arriving?
- Walk through what happens when an ACK is lost but the data arrived.
- What is Karn's algorithm protecting against? Give a concrete example.
- Change the window from 32 to 8 while a transfer is running.
- Add a new packet type to the protocol, right now.

11. Before we submit the proposal — ask sir

- The brief PDF looks truncated: it refers to "Section 6" for individual moderation and "Section 7" for rules on scanning networks, but the document ends at Section 6, the proposal template. Is there a fuller version?
- No proposal deadline or submission method is stated anywhere in the document.

- The header says team size 4 strictly, but the Path B bar describes scope as "roughly 8 weeks for three people." Which applies to us?
